Design of a Projected-Maze Line-Following Robot: Phototransistor Sensing, Motor Control and Shortest-Path Planning

Alistair Campbell, Disan Tennakoon, Dohyun Kim and Sharun Anton Constantine
COMPSYS 301, Group 12
Department of Electrical, Computer and Software Engineering, The University of Auckland, New Zealand

***Abstract*—This report describes a two-wheeled robot that follows a maze projected onto the floor and travels between specified locations by the shortest route. A phototransistor sensor channel was designed around the 60 Hz projector flicker: a common-collector stage with a 47 kΩ sensing resistor, a 3.1 Hz high-pass filter, a 2 V offset and unity-gain buffers feed a 12-bit ADC sampled at 4 kHz, and white and black are separated in software by the peak-to-peak amplitude over a 100-sample window. Five pathfinding algorithms were compared, and A* and breadth first search (BFS) were implemented and tested against each other on two maps. Both returned identical shortest paths, but BFS was chosen for the PSoC because of its simplicity and memory use. Motor control combines feed-forward and PI speed loops with encoder-held heading steering. Benchmark and final demonstration results are analysed, including the faults that limited turning reliability.**

***Index Terms*—line following, phototransistor, high-pass filter, PSoC, breadth first search, A*, PI control**

I. Introduction

The robot must detect and follow lines about 15 mm wide projected onto the floor by a ceiling-mounted projector, and travel between specified start and end locations by the shortest possible route. The project has a hardware part and a software part. The hardware part is a phototransistor sensor circuit and a printed circuit board (PCB) that mounts six sensors on the provided robot chassis. The software part is C firmware on a PSoC microcontroller that processes the sensor signals, controls the motors and runs the pathfinding. Sections II to V cover the design considerations and decisions for the sensor, pathfinding, motor control and PCB. Section VI presents verification, benchmark and final demonstration results.

II. Sensor Design

A. Requirements and projector behaviour

The sensor must tell a white region from a black region of the projected map, give a large and repeatable difference between them, stay inside the 0 to 5 V supply without saturating, reject ambient light, reflections and shadows, and respond quickly enough for the robot to react.

The projector does not emit steady light. It scans the image, so a stationary sensor sees a burst of light that repeats mainly at 120 Hz, twice the projector's 60 Hz refresh rate. These appear as a swing between a minimum and a maximum in the measured output (Table I). The spectrum of the measured sensor output under white light, computed as an FFT in LTspice (Fig. 1), shows harmonics of 60 Hz up to about 800 Hz, with the strongest peaks at 120 Hz and 240 Hz. The fundamental frequency is therefore 60 Hz, set by the projector refresh rate, but the strongest component is at 120 Hz. By 2 kHz the content is about 45 dB below the largest peak, and it sits on a DC level from ambient light. White and black are therefore told apart by the size of this flicker rather than by an absolute voltage range.

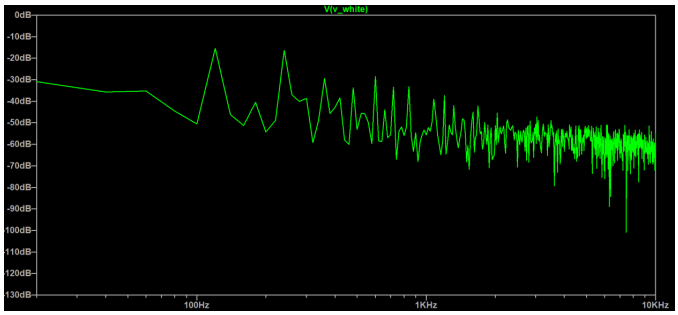


Fig. 1. FFT of measured sensor output under white light (computed in LTspice).

B. Configuration and sensing resistor selection

The VEMT3700 phototransistors can be wired as a common-collector (CC) or common-emitter (CE) stage. Light sets the collector current I_c, which flows through the sensing resistor R_s. In CE, R_s sits at the collector and the output is $V_{cc} - I_c R_s$, so noise on the 5 V rail passes straight onto the signal. In CC, R_s sits at the emitter and the output is $I_c R_s$ referenced to ground, so it follows the light and largely ignores the rail. This supply-noise rejection was the main reason for choosing CC. CC also rises with light, which is convenient but not essential because peak-to-peak detection works with either polarity.

TABLE I
CC OUTPUT RANGE (V) FOR EACH SENSING RESISTOR

R_s	White, lights on	Black, lights on	White, lights off	Black, lights off
10k	0.2 to 0.6	0.2 to 0.4	0 to 0.68	0 to 0.07
15k	0.4 to 1.2	0.2 to 0.6	0 to 0.51	0 to 0.11
27k	0.8 to 2.16	0.8 to 1.2	0 to 0.76	0 to 0.12
47k	1.4 to 3.41	1.4 to 2.0	0.11 to 1.6	0 to 0.23
100k	3.11 to 4.82	2.6 to 3.6	0.44 to 2.44	0.15 to 0.51

Table I gives the minimum and maximum of the CC output, read from an oscilloscope, for R_s from 10 kΩ to 100 kΩ with the room lights on and off to measure the effect of ambient light. Larger resistors give a larger swing because the same photocurrent develops more voltage, so the largest resistor that avoids saturation is preferred. Ambient light adds a large DC offset (the 47k minimum rises from about 0.1 V with the lights off to 1.4 V with them on), so the lit room is the worst case. At 100k the white output reached 4.82 V, close to the 5 V rail. Its swing fell to 1.71 V while black rose to 1.0 V, so the transistor was saturating and contrast dropped. At 47k with the lights on, white swung 2.0 V peak-to-peak (1.4 to 3.41 V) against 0.6 V for black (1.4 to 2.0 V). This gap is what detection relies on, it was the clearest of any resistor, and the peak stays well below the rail. 47k was therefore chosen.

C. High-pass filter

A high-pass filter removes the ambient DC level. The fundamental is 60 Hz, so the cut-off was targeted at least a decade below it, under 6 Hz. The transistor acts as a current source, so the filter sees about R_s as source impedance in series with the filter resistor, forming a divider. To limit insertion loss the filter resistor is ten times R_s, $R_f = 470$ kΩ. With $C_f = 100$ nF the cut-off is

$$f_c = \frac{1}{2\pi(R_f + R_s)C_f} = \frac{1}{2\pi(517\ \text{k}\Omega)(100\ \text{nF})} \approx 3.1\ \text{Hz}. \tag{1}$$

A 56 nF capacitor would give about 5.5 Hz, but 100 nF keeps the cut-off below 6 Hz once tolerances are included. The gain at 60 Hz, combining insertion loss and the high-pass response, is

$$|H(60)| = \frac{R_f}{R_f + R_s} \cdot \frac{60/f_c}{\sqrt{1 + (60/f_c)^2}} \approx 0.91 \times 0.999 \approx 0.91. \tag{2}$$

The flicker therefore passes with a 9% loss while DC is blocked. The LTspice response in Fig. 3(a) agrees: the gain rises with frequency, levels off by about 10 Hz and stays flat across the flicker band. The full channel is shown in Fig. 2. A low-pass filter was considered while designing the comparator (Section II-E), where noise caused false triggers near the threshold. It was left out of the final design because white sits well above the threshold and black well below it.

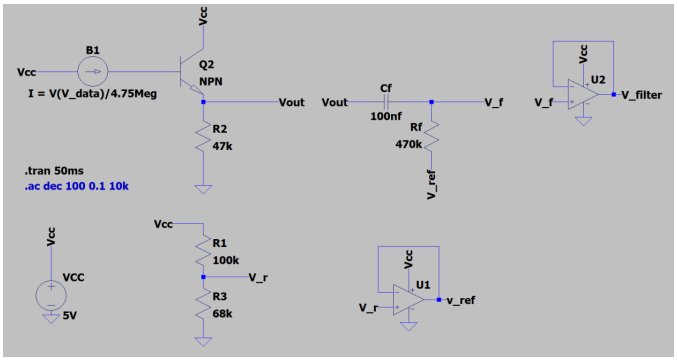


Fig. 2. One sensor channel (CC phototransistor, R_s, C_f, R_f, output buffer) and the shared buffered reference.

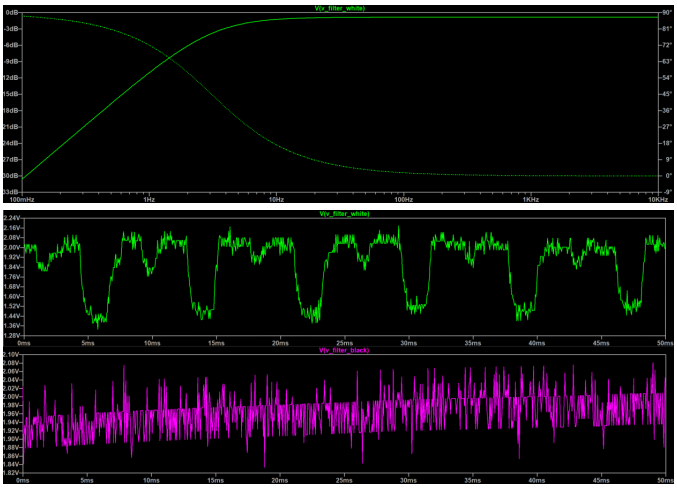


Fig. 3. (a) Simulated high-pass filter response (gain solid, phase dotted). (b) Simulated filtered output under white light (top) and black light (bottom) using real data.

D. Offset

The high-pass filter centres the signal on 0 V, so it swings negative, but the ADC input range is 0 to 5 V. The filter resistor is therefore tied to a reference from a divider on the 5 V supply. A first divider of 100k and 47k gave 1.6 V, but white light still drove the signal to about −200 mV on the breadboard. Raising the lower resistor to 68k gave

$$V_{ref} = 5 \times \frac{68\ \text{k}\Omega}{100\ \text{k}\Omega + 68\ \text{k}\Omega} \approx 2.0\ \text{V}. \tag{3}$$

This lifts the white-light minimum to about +200 mV and leaves roughly 2 V of headroom below the reference and 3 V above it. Fig. 3(b) shows the result: the white output shows a clear flicker, dipping to about 1.35 V roughly every 8 ms, while the black output stays near 2 V with only small noise. This amplitude difference is what the detector measures.

E. Detection: why ADC instead of a comparator

A comparator with a pull-up to 5 V was designed first. In LTspice, 0.1 V of hysteresis gave false triggers from noise while 0.3 V was stable but slower, so the hardware approach trades noise immunity against speed. Software detection avoids

this: the threshold can be changed without swapping components, and the peak-to-peak amplitude over a window ignores single noise spikes whereas a comparator reacts to every crossing. The physical comparator also could not be made reliable in the time available, so detection moved to the ADC.

The filtered signal is sampled by the PSoC 12-bit SAR ADC. A hardware timer triggers each conversion, so the sample rate, window length and Nyquist frequency are fixed and known. An initial timer period of 4000 gave 250 Hz, which would alias the 60 to 800 Hz signal. A 1 MHz clock with a period of 250 gives $f_s = 4$ kHz and a Nyquist frequency of 2 kHz. With no low-pass filter, energy above 2 kHz could fold back, but the spectrum is about 45 dB below its largest peak there, and the threshold margin covers the small error this adds. The ADC sequencer converts all six channels on each trigger, so every channel is sampled at 4 kHz.

Samples fill a circular buffer that overwrites the oldest sample, so a full window is always available. The window lasts $T = N/f_s$ and must contain at least one 60 Hz period (about 67 samples) while staying short for fast response. A 250-sample window (62.5 ms) was too slow, so $N = 100$ was used, giving 25 ms or 1.5 periods. The code compares the difference between the maximum and minimum samples with a threshold, which ignores the DC offset. With a 5 V reference one count is 1.22 mV. Black measured 400 to 680 counts and white 2000 to 2350 counts, so the threshold was set between them at 1000 counts (1.22 V), leaving margins of 320 and 1000 counts.

F. Buffers

Unity-gain op-amp buffers have high input and low output impedance and isolate each stage without changing the signal. The V_{ref} divider has an output impedance of about 40 kΩ, so current drawn by the six filters would make the reference sag and couple flicker between channels. A single buffer therefore holds V_{ref} at 2 V for all six. Each filter output is also buffered before the ADC: the sample-and-hold capacitor C_s charges through the source resistance with time constant $\tau = R_{src}C_s$, and the high source impedance of the filter could prevent it settling to within half an LSB before conversion, reducing accuracy. The buffers are LM324 op-amps on a single 5 V supply. Their output only reaches about 3.5 V, but the filtered signal peaks near 3.1 V so it stays in range.

III. PATHFINDING

The pathfinding must find the shortest valid route between a start and end point on a 2D unweighted grid maze, without crossing walls or leaving the grid. Five algorithms were compared (Table II), with their general characteristics drawn from [4].

TABLE II
CANDIDATE PATHFINDING ALGORITHMS

Algorithm	Shortest path?	Memory	Main limitation for this project
A*	Yes	Highest	Complex; benefit needs large open maps
BFS	Yes	High	Explores level by level (ripple)
DFS	No	Lowest	May explore long routes before short ones
Greedy BFS	No	Low	Detours and traps at dead ends
Dijkstra	Yes	Very high	Explores all directions; pointless on unweighted map

Dijkstra's algorithm [3] was dropped because on an unweighted map BFS gives the same result more simply. Greedy best first search was dropped because the maps contain many intersections and dead ends and it cannot guarantee the shortest path, a core requirement. DFS was the closest alternative for its low memory and simplicity, but it follows a fixed neighbour order and may find a longer route first, so it was rejected in favour of guaranteed optimality. A* and BFS were selected for implementation and testing.

A. A search*

A* [1] ranks nodes by $f(n) = g(n) + h(n)$, where $g(n)$ is the steps taken from the start and $h(n)$ estimates the steps remaining. Manhattan distance, $h(n) = |r - r_{goal}| + |c - c_{goal}|$, was used. Walls are handled when neighbours are checked, and since the robot moves in four directions with no diagonals the heuristic never overestimates, so the path is optimal. Euclidean distance is always smaller or equal, a looser estimate that tends to explore more nodes. The implementation is described in Appendix B.

B. Breadth first search

BFS [2] expands cells level by level from the start using a queue, which guarantees the shortest path on an unweighted graph. It stores a visited flag and a parent for each cell, and rebuilds the route by tracing parents back from the target (Appendix B, Fig. 6). Its drawback is the memory needed for the waiting cells, accepted here because the maze is small. A DFS version had been written in MATLAB first, and about 85% of that code was reused when the group switched to BFS, so the change was quick. Fig. 4 shows the BFS route found on one map.

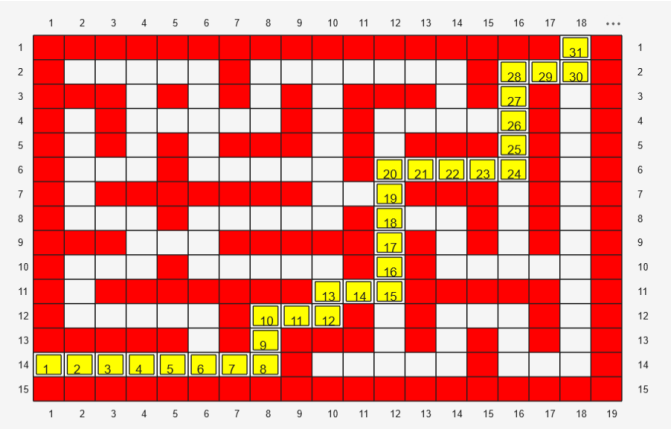


Fig. 4. MATLAB BFS implementation: shortest route (30 steps, numbered cells) from [14,1] to [1,18] on Map 8.

C. Testing and selection

Table III shows that A* and BFS returned identical path lengths on all six routes, while A* always explored fewer nodes. On Map 8 the saving was often large (A*/BFS ratio 0.31 to 0.64), while on Map 1 it was small (0.83 to 0.96). Heuristic search helps most on open maps where the route is close to the straight-line distance, which the detour column confirms: Map 1 routes had long detours of 8 to 72 steps over Manhattan distance. Further tests on 20 random start and end pairs per map (seed 1) gave mean A*/BFS node ratios of 0.83 on Map 1 and 0.35 on Map 8 with no path length mismatches. Map 1 has no loops, while Map 8 has 10, consistent with the longer detours bringing A* closer to BFS.

TABLE III
A* VERSUS BFS ON THE TWO TEST MAPS (NODES EXPLORED)

Map	Route	Steps	Manh.	Detour	A*	BFS	A*/BFS
8	[14,1] to [1,18]	30	30	0	42	134	0.31
8	[5,16] to [4,8]	13	9	4	33	71	0.46
8	[7,18] to [4,2]	29	19	10	87	136	0.64
1	[2,2] to [4,18]	26	18	8	26	27	0.96
1	[6,10] to [10,12]	30	6	24	60	72	0.83
1	[6,2] to [2,2]	76	4	72	112	122	0.92

The first A* implementation had a subtle flaw: it did not update g, f and the parent when a shorter route to an open node was found. It returned a 27-step path from [14,17] to [8,2] where the true shortest was 25 steps, found with the independent MATLAB BFS function. After the fix both algorithms agreed on all 46 coordinate pairs (6 in the table and 40 random).

BFS is also far simpler: it needs only a visited flag and a queue, while A* adds an update step, lowest-f selection and an admissible heuristic. The maps are only 15×19 cells (285 cells), so both finish within milliseconds and A*'s advantage is small in practice. The BFS state used about 9.1 kB of PSoC RAM and A* needs at least that plus the open list and f values, and other teams reported memory problems with A* on the robot. BFS in C was therefore chosen as the final algorithm, with static memory allocation, since the efficiency of A* did not outweigh BFS's simplicity and lower memory. A tie-break prefers the route with fewer turns, using the robot heading, because turning was the main source of error in BT1.

IV. Motor Control and Steering

A. Drive and encoders

Both motors use 8-bit PWM in locked anti-phase mode, so a compare value of 128 gives 50% duty and the motor is stationary. The measured deadbands were 38 counts (M1) and 33 counts (M2), so the percent-to-compare conversion skips this region and a moving wheel is never commanded below 1%. Without this the controller occasionally output 0% and the jump back into the deadband made the motors stutter. Quadrature decoding at $4\times$ with a 19:1 gearbox gives 228 counts per revolution. The wheel diameter was first entered as 95 mm instead of 65 mm, which was exposed when a 1000 mm run covered only 690 mm. Each count is now $\pi \times 65/228 = 0.896$ mm. Wheel slip during pivots means a full real turn reads only about 295 encoder degrees on the 144 mm track width, so a turn scale of 0.82 was added.

B. Speed control

Each wheel has its own controller sampled every 50 ms. A feed-forward term maps target speed to duty cycle for fast response, and a PI term removes the mismatch between the motors. A line through zero asked for too little duty at low speed because of the deadband, so the feed-forward was replaced by a fit to field logs (about 95 mm/s at 1% plus 5 mm/s per extra percent). The proportional gain was cut from 1.5 to 0.8 because it reacted to single-count encoder jitter. Logs showed the robot running 10 to 30% above target after each turn, so the integral gain and limit were lowered from 0.2 and 40% to 0.12 and 25%. Measured speed is averaged over three samples (150 ms) and speed changes are ramped at 250 mm/s^2.

C. Steering and corners

Steering directly from the rear sensors D4 and D5 made the robot weave, because they sit 5 mm behind the axle and only see the line once the robot is about 10 mm off centre. Heading control was adopted instead: the sensors choose a target heading and the encoders hold it. Sensor D2 follows the left edge of the line, aiming 5 degrees to either side and growing to 8 degrees if it stays on one side, so steering is gentle on track and firm when drifting. D4, D5, D1 and D3 act as a safety net with stronger corrections.

After D1 or D3 detects a branch, the robot drives on before pivoting. Geometry gave 88 mm (front sensor row to axle), but the robot rolls on after the motors stop, so this was reduced to 65 mm on the practice map and 70 mm for unseen maps. A fixed 90 degree encoder angle overturned on the real robot, so the final rule uses the calibrated angle (74 encoder degrees), then a short line search and slow settle if D2 is not on the line. Field logs showed the pivot centre sits about 15 mm right of the axle, so right pivots start 8 mm early and left pivots

8 mm late. Speeds are in Table IV. Below about 100 mm/s performance worsened, since one encoder count becomes a 20% speed error and low duties fall in the deadband. The deadband also shifts with battery charge, so tuning and demos should use a similar charge.

TABLE IV
CHOSEN SPEEDS

Setting	Speed (mm/s)	Reason
Task 2	450	Most stable speed tested (61% of maximum)
Maze straights	150	Holds the line with edge following
Approach and corners	110	D1 and D3 must not miss thin branches
Pivot	130, then 100	Slower falls below the motor floor of about 95

V. PCB DESIGN

The robot is asymmetrical, so six phototransistors were arranged as in Fig. 5 (Appendix A). Three sit in a row at the front, 88 mm ahead of the axle: one on the centre line and one 36.4 mm to each side. The side sensors stay off the 15 mm line during following and only see it when a branch appears, which is how intersections are detected. The front centre sensor follows one edge of the line and a middle sensor 30 mm ahead of the axle confirms the robot is still on it. Two sensors near the axle sit 14.7 mm apart, just wider than the line, so both see white when aligned and one goes dark when the robot drifts.

Each sensor has a green debug LED with a 100 Ω resistor that turns off over the line, and all six blink a stop code at the end of a run. Six 100 nF decoupling capacitors sit near the op-amp buffers and power connectors. Test points on every sensor channel and on the 5 V, ground and 2 V offset lines allow each signal to be probed with an oscilloscope.

VI. TESTING AND RESULTS

A. Benchmark Test 1 (turns)

Preparation was rushed and turns were inconsistent, giving 1 to 6 successful runs per ten. Across five BT1 runs the robot averaged about five successful turns out of eleven. The main fault was overturning at intersections: small misalignment after the first turns grew with each turn until the robot left the line, the sensors lost the path and it stopped. The robot also randomly stopped and scanned one side before resuming straight. After the first three runs there was time to adjust, but the exact code fault could not be found, so the turning speed was reduced. One of the last two runs then completed ten turns and the other about five. Lower turning speed helped but did not remove the cause, which led to the calibrated pivot angle and heading control of Section IV.

B. Benchmark Test 2 (odometry)

BT2 assessed movement control and odometry: travel a straight line at a set speed, stop at a given distance, and compare expected and actual distance and time. The robot was set to a 450 mm/s maximum, and the aim was to minimise percentage error in stopping distance. Preparation was limited: distance was measured at several speeds, but travel times were not consistently recorded, so little data existed to verify speed and distance calculations beforehand. The robot still performed considerably better than in BT1, because BT2 needs only simple straight motion while BT1 needs repeated turns and accurate line detection at intersections. The 690 mm versus 1000 mm result (Section IV-A) shows why encoder calibration mattered.

C. Final demonstration

The robot had to travel from the start through five target locations (foods) by the shortest path. BFS replaced A* because of problems running the A* code, and the MATLAB BFS was ported to C with static memory allocation. The first two runs reached only one of five targets, because the robot repeatedly failed to reach the second. The markers allowed the second target to be dropped, so the robot navigated through the first, third, fourth and fifth, at a cost of one mark per later run while keeping one out of five for the first run. In runs three to five the robot reached all four remaining targets, but still struggled with sharp corners and ran slowly. Afterwards, battery drain and turn overshoot were suspected, but repeated debugging could not isolate the cause because results stayed inconsistent.

VII. CONCLUSION

A two-wheeled robot was designed and built to follow a projected maze by the shortest route. The sensor channel exploits the 60 Hz projector flicker: a 47 kΩ CC stage, 3.1 Hz high-pass filter, 2 V offset and buffers feed a 4 kHz ADC, and a 100-sample peak-to-peak window gave 320 and 1000 counts of margin to the threshold. A* and BFS found identical shortest paths on 46 test routes, and BFS was kept for its simplicity and lower memory. Turning accuracy, line detection at intersections and movement consistency were the main problems, visible in BT1 and improved by encoder calibration and heading control before BT2 and the final demonstration, where all four remaining targets were reached in runs three to five. Overall, finding the shortest path was only one part of the successful navigation, as accurate movement control and reliable sensor detection were equally important. Further improvements to turning accuracy, battery performance and software debugging would improve the robot's overall reliability.

REFERENCES

[1] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100-107, 1968.

[2] E. F. Moore, "The shortest path through a maze," in *Proc. Int. Symp. Theory of Switching*, Harvard Univ. Press, 1959, pp. 285-292.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269-271, 1959.

[4] F. Rubio, "Pathfinding algorithms: top 5 most powerful," Graphable, Jun. 6, 2023. [Online]. Available: https://graphable.ai/blog/pathfinding-algorithms/

APPENDIX A
PCB LAYOUT

Fig. 5 shows the final board layout described in Section V.

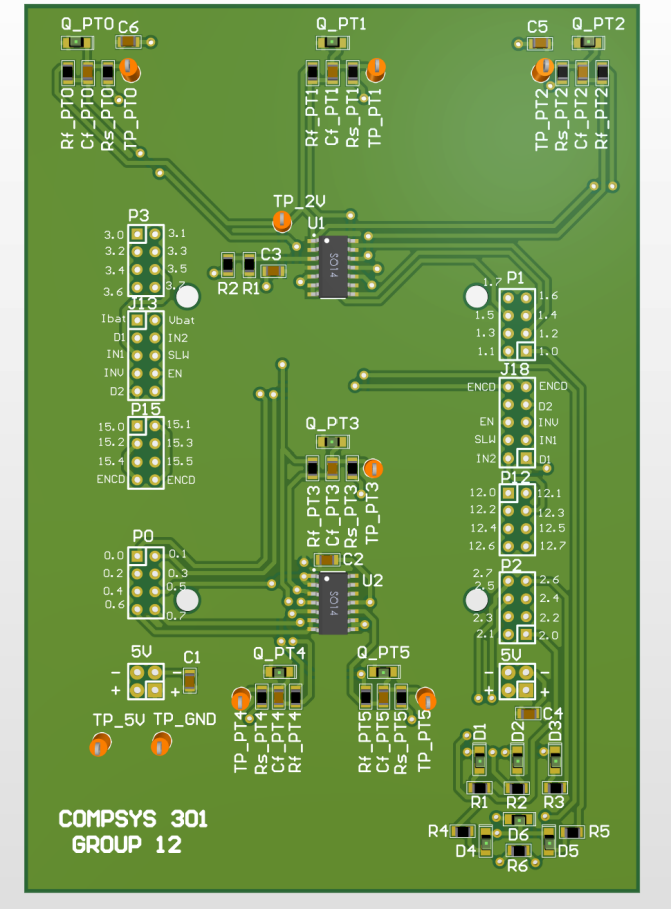


Fig. 5. PCB layout: six phototransistor channels (Q_PT0 to Q_PT5) with R_s, C_f and R_f, two buffer ICs (U1, U2), test points and debug LEDs D1 to D6.

APPENDIX B
PATHFINDING IMPLEMENTATION DETAILS

A* implementation. The map is a matrix of zeros (open) and ones (walls). Two parent matrices initialised to -1 record the row and column of each cell's parent. An open list holds one row per candidate cell [row, column, g, f], and a closed list holds explored nodes. The start is placed in the open list with $g = 0$. The node with the lowest f is chosen. If it is the target, the route is traced back through the parent matrices. Otherwise it moves to the closed list and each of its four neighbours is skipped if it is a wall, outside the map or closed, else added to the open list with $g + 1$, parent set to the current node and $f = g + h$. If a new route to an open node has a lower g, its g, f and parent are replaced. If the open list empties, the target is unreachable and an empty path is returned. For route [2,2] to [4,18], h at the start is $|2 - 4| + |2 - 18| = 18$.

BFS code structure. Table V lists the code order and Fig. 6 the flowchart.

TABLE V
BFS CODE STRUCTURE

Stage	Explanation
Setup	Loads the maze from the provided files
Trackers	`visited` marks explored cells, `parent` stores the previous cell
Start queue	Adds the starting location to the queue
Main loop	Explores cells in the order they entered the queue
Neighbours	Checks right, up, down, left, ignoring walls and visited cells
Find target	Stops once the target cell is reached
Reconstruct	Traces `parent` back from target to start
Return	Returns maze, visited cells and shortest path

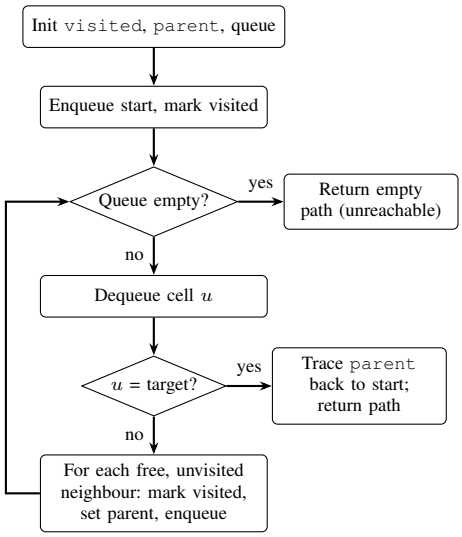


Fig. 6. BFS flowchart.

APPENDIX C
TEAM CONTRIBUTIONS

Table VI lists each member's main project contributions, the report sections they drafted and their estimated total hours.

TABLE VI
CONTRIBUTIONS AND ESTIMATED HOURS PER MEMBER

Member	Project work	Report sections	Hours
Alistair Campbell	Sensor design, BT2, PCB, debugging	Sensor, PCB	180
Disan Tennakoon	A* pathfinding, debugging	Pathfinding	100
Dohyun Kim	Sensor design, motor control, BT1, pathfinding implementation	Motor control, BT, conclusion	180
Sharun Anton Constantine	BFS pathfinding, PCB, pathfinding implementation	Abstract, pathfinding, PCB, conclusion	140